Process & Decision Documentation

This document is used to make your design and development process visible. At this stage of your academic career, you are expected not only to produce finished work, but to articulate how decisions were made, how ideas changed, and how collaboration (for the assignments that include group work) unfolds

In professional and co-op contexts, employers do not only evaluate your final projects in your portfolio. They often ask candidates to explain their process, justify trade-offs, reflect on iteration, and describe their roles within a team.

You will need to submit a modification of this document for every group assignment (A1 – A3) and a shorter version for your individual assignments (Side Quests and A4).

For A1 – A3, this is a group document submitted once per group. Each group member must clearly document their own role and responsibilities. Different roles will naturally produce different design processes.

Side Quests and A4 (Individual Work)

Keep this section brief, typically 2 to 4 sentences.

Focus on:

- One significant decision or change you made
- Why you made it
- What effect it had on the work

Examples:

- Simplifying a mechanic so it functioned correctly
- Changing an approach after something failed
- Deciding not to pursue an idea due to time or technical limitations

You are not expected to document every alternative or iteration

Entry Header

Name: Ria Anand

Role(s): Designer, Programmer

Primary responsibility for this work: Coding the interactive story that unfolds through multiple game states and files.

Goal of Work Session

Make an interactive story game that could unfold with multiple pages and interactions. Also add a bar that tracks trust level throughout story unfolding.

Tools, Resources, or Inputs Used

- GenAI tool: ChatGPT (GPT -5.2)
- Lecture concepts
- Self-testing in browser (manual playtesting)

GenAI Documentation

Everyone must complete this section. If not GenAI was used, write, "No GenAI used for this task." When GenAI is not used, process evidence should still demonstrate iteration, revision, or development over time.

Because GenAI can closely mimic human-created work, instructors or TAs may occasionally request additional process evidence to confirm non-use. This may include original working files (e.g., an illustrator file), intermediate drafts, or a brief check-in with a TA to walk through your process.

These requests are not an assumption of misconduct. They are part of ensuring academic integrity in an environment where distinguishing between human-created and AI-generated work is increasingly difficult.

If GenAI was used (keep each response as brief as possible):

Date Used: February 2, 2026

Tool Disclosure: ChatGPT 5.2

Purpose of Use: Writing and editing code, game description and files naming/organization.

Summary of Interaction: The tool was used to generate a interactive story framework, including a decision-tree structure, basic game logic, stat tracking, and a coherent narrative. It sped up the technical setup and provided a baseline design that could be used for this side quest.

Human Decision Point(s): I had to remake the initial multi-file game due to multiple interaction failures and restarted the project from scratch. I redirected the tool to simplify the game, reduce any technical risks, and better align the game with course expectations.

Integrity & Verification Note: I tested the game logic manually by clicking through all branches to confirm state changes, stat updates, and endings were as intended. I also asked the tool to make the game relevant to course content which led to the narrative of the game to be about course concepts such as values, consequences, and ethical decision-making.

Scope of GenAI Use: GenAI did not determine the final narrative focus, ethical framing, or how the game would be positioned within the assignment. It did not write the reflective components, analysis, or course-specific explanations. Structural decisions about what to keep, cut, or simplify were made alone.

Limitations or Misfires: The tool initially produced over-engineered code that failed in live testing and required complete restructuring. It could not account for my local development environment or detect interaction bugs without manual testing.

Summary of Process (Human + Tool)

Describe what you did, focusing on process rather than outcome. This may include:

I worked through several iterations using both GenAI and hands-on testing. I initially relied on the tool to generate a more complex interactive story structure, but once I began testing it, I ran into repeated technical issues that prevented the game from progressing correctly. After attempting to fix these problems, I decided to restart the project with a simpler structure. From there, I moved back and forth between using the tool for logic support and manually refining the narrative, testing each change to make sure the game functioned as intended.

Decision Points & Trade-offs

Describe one or two key decisions you made:

One major decision was choosing to abandon the original version of the game and rebuild it from scratch. While fixing the original code was an option, it would have taken significant time and distracted from the assignment's focus on decision-making and values. Simplifying the system reduced technical complexity but made the decision points clearer and more reliable. Another key choice was narrowing the story to a single ethical dilemma, which better aligned with the decision-tree visualization required for the assignment.

Verification & Judgement

Explain how you evaluated whether your decision or change was appropriate:

I evaluated my decisions through repeated playtesting, clicking through every possible branch to ensure choices led to different outcomes and stat changes. I also revisited the assignment criteria to confirm that the game clearly demonstrated meaningful decisions and consequences. Throughout the process, I compared the narrative and mechanics to course concepts around ethics and responsibility to ensure the game encouraged reflection rather than optimization.

Limitations, Dead Ends, or Open Questions

The original version of the game did not function properly and had to be discarded. Some planned branches and mechanics were removed to keep the project stable and manageable. The trust stat is intentionally simplified and does not capture the full complexity of ethical behaviour.

Appendix

Please include a full transcript of your conversation with the GenAI. GenAI transcripts are used for transparency and verification only and are not assessed for writing quality or completeness.

Note: If GenAI is used in another language, you must include both the original language and a translated transcript in the appendix

Ria

Build an interactive story that unfolds through

multiple game states and files, branching like a small

decision tree.

• Bonus: Track a player stat (e.g., trust, health, karma)

across scenes and unlock endings based on it.

Chat

File tree

```
project/
  index.html
```

```
  style.css
  main.js
  stateManager.js
  data.js
  utils.js
  ui.js
  scenes/
    title.js
    scene1.js
    scene2.js
    scene3.js
    endings.js
```

---

index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width,initial-scale=1" />
    <title>Branching Story (p5.js)</title>
    <link rel="stylesheet" href="style.css" />
    <script
src="https://cdn.jsdelivr.net/npm/p5@1.9.0/lib/p5.min.js"></script>

    <!-- Core -->
    <script defer src="data.js"></script>
    <script defer src="utils.js"></script>
    <script defer src="ui.js"></script>
    <script defer src="stateManager.js"></script>

    <!-- Scenes -->
    <script defer src="scenes/title.js"></script>
    <script defer src="scenes/scene1.js"></script>
    <script defer src="scenes/scene2.js"></script>
    <script defer src="scenes/scene3.js"></script>
    <script defer src="scenes/endings.js"></script>

    <!-- p5 entry -->
    <script defer src="main.js"></script>
  </head>
  <body>
    <main id="app"></main>
  </body>
</html>
```

style.css

```
html, body {
  margin: 0;
  padding: 0;
  background: #0b0d12;
  color: #e9eefc;
  font-family: system-ui, -apple-system, Segoe UI, Roboto, Arial, sans-serif;
}
```

```
canvas {
  display: block;
  margin: 0 auto;
  outline: none;
}
```

---

data.js

```
// Global game model
let game = {
  state: "TITLE",
  karma: 0,          // player stat tracked across scenes
  flags: {
    helpedStranger: false,
    liedToGuard: false,
    tookBribe: false,
    savedFriend: false,
  },
  log: [],           // optional: record choices
};

const STATES = {
  TITLE: "TITLE",
  S1: "SCENE_1",
  S2A: "SCENE_2A",
  S2B: "SCENE_2B",
  S3A: "SCENE_3A",
  S3B: "SCENE_3B",
  END: "ENDING",
};

function resetGame() {
  game.state = STATES.TITLE;
  game.karma = 0;
  game.flags = {
    helpedStranger: false,
    liedToGuard: false,
    tookBribe: false,
    savedFriend: false,
  };
  game.log = [];
}
```

---

utils.js

```
function clamp(v, lo, hi) {
  return Math.max(lo, Math.min(hi, v));
}

function isHover(rect) {
  return (
    mouseX >= rect.x &&
    mouseX <= rect.x + rect.w &&
```

```
    mouseY >= rect.y &&
    mouseY <= rect.y + rect.h
  );
}

function addKarma(delta, reason) {
  game.karma += delta;
  game.log.push({ t: Date.now(), delta, reason });
}
```

---

ui.js

```
function drawPanel(x, y, w, h) {
  push();
  noStroke();
  fill(18, 22, 33, 235);
  rect(x, y, w, h, 16);
  pop();
}

function drawHeader(title, subtitle) {
  push();
  fill(233, 238, 252);
  textAlign(LEFT, TOP);
  textStyle(BOLD);
  textSize(28);
  text(title, 36, 28);
  textStyle(NORMAL);
  textSize(14);
  fill(185, 196, 220);
  text(subtitle, 36, 62);
  pop();
}

function drawStatBar() {
  // Karma range visualization: -5..+5 (clamped)
  const k = clamp(game.karma, -5, 5);
  const x = 36, y = 92, w = 360, h = 14;

  push();
  noStroke();
  fill(255, 255, 255, 18);
  rect(x, y, w, h, 8);

  const mid = x + w / 2;
  const px = map(k, -5, 5, x, x + w);

  // negative fill
  fill(255, 120, 120, 140);
  rect(Math.min(px, mid), y, Math.abs(px - mid), h, 8);

  // positive fill
  fill(120, 255, 190, 140);
  rect(Math.min(px, mid), y, Math.abs(px - mid), h, 8);
```

```
  // center line
  stroke(255, 255, 255, 40);
  line(mid, y - 2, mid, y + h + 2);

  noStroke();
  fill(233, 238, 252);
  textSize(12);
  textAlign(LEFT, BOTTOM);
  text(`KARMA: ${game.karma}`, x, y - 6);
  pop();
}

function buttonRect(cx, cy, w, h) {
  return { x: cx - w / 2, y: cy - h / 2, w, h };
}

function drawButton(label, cx, cy, w, h, hot) {
  const r = buttonRect(cx, cy, w, h);
  const hovering = isHover(r);

  push();
  noStroke();
  if (hot || hovering) fill(90, 120, 255, 210);
  else fill(255, 255, 255, 18);
  rect(r.x, r.y, r.w, r.h, 14);

  fill(233, 238, 252);
  textAlign(CENTER, CENTER);
  textSize(14);
  text(label, cx, cy);
  pop();

  return { ...r, hovering };
}

function drawBodyText(lines, x, y, w, lineH = 22) {
  push();
  fill(233, 238, 252);
  textAlign(LEFT, TOP);
  textSize(16);

  let yy = y;
  for (const line of lines) {
    text(line, x, yy, w);
    yy += lineH;
  }
  pop();
}
```

---

stateManager.js

```
function setState(next) {
  game.state = next;
}
```

```
function currentScene() {
  switch (game.state) {
    case STATES.TITLE: return TitleScene;
    case STATES.S1:    return Scene1;
    case STATES.S2A:   return Scene2A;
    case STATES.S2B:   return Scene2B;
    case STATES.S3A:   return Scene3A;
    case STATES.S3B:   return Scene3B;
    case STATES.END:   return EndingsScene;
    default:           return TitleScene;
  }
}

function sceneDraw() {
  currentScene().draw();
}

function sceneMousePressed() {
  const s = currentScene();
  if (s && typeof s.mousePressed === "function") s.mousePressed();
}
```

---

main.js

```
function setup() {
  createCanvas(900, 520);
  textFont("system-ui");
  noSmooth();
}

function draw() {
  background(11, 13, 18);

  // subtle grid
  push();
  stroke(255, 255, 255, 10);
  for (let x = 0; x < width; x += 24) line(x, 0, x, height);
  for (let y = 0; y < height; y += 24) line(0, y, width, y);
  pop();

  sceneDraw();
}

function mousePressed() {
  sceneMousePressed();
}
```

---

scenes/title.js

```
const TitleScene = {
  draw() {
```

```
    drawHeader("THE SILENT STATION", "A branching story with a tracked stat
(karma).");
    drawStatBar();

    const panel = { x: 36, y: 130, w: 828, h: 310 };
    drawPanel(panel.x, panel.y, panel.w, panel.h);

    const lines = [
      "You arrive at a closed transit station after curfew.",
      "A guard watches the gate. A stranger sits nearby, shivering.",
      "Every decision shifts karma. Endings unlock based on karma and
choices.",
      "",
      "Goal:",
      "Reach the platform and decide what kind of person you are under
pressure."
    ];
    drawBodyText(lines, panel.x + 24, panel.y + 22, panel.w - 48);

    this.btnStart = drawButton("START", width / 2, 460, 240, 44, true);
    this.btnReset = drawButton("RESET", 770, 46, 120, 34, false);
  },

  mousePressed() {
    if (this.btnStart?.hovering) {
      setState(STATES.S1);
    }
    if (this.btnReset?.hovering) {
      resetGame();
    }
  },
};
```

---

scenes/scene1.js

```
const Scene1 = {
  draw() {
    drawHeader("SCENE 1: OUTSIDE THE GATE", "First fork. Your first move sets
the tone.");
    drawStatBar();

    const panel = { x: 36, y: 130, w: 828, h: 310 };
    drawPanel(panel.x, panel.y, panel.w, panel.h);

    const lines = [
      "The guard doesn’t acknowledge you.",
      "A stranger whispers: “My phone’s dead. I just need to call someone.”",
      "",
      "What do you do?"
    ];
    drawBodyText(lines, panel.x + 24, panel.y + 22, panel.w - 48);

    this.b1 = drawButton("Help the stranger (lend phone / call)", width / 2,
320, 520, 46, false);
```

```
    this.b2 = drawButton("Ignore and approach the guard", width / 2, 380,
520, 46, false);
    this.back = drawButton("TITLE", 820, 46, 120, 34, false);
  },

  mousePressed() {
    if (this.b1?.hovering) {
      if (!game.flags.helpedStranger) {
        game.flags.helpedStranger = true;
        addKarma(+2, "Helped the stranger");
      }
      setState(STATES.S2A);
    }
    if (this.b2?.hovering) {
      addKarma(-1, "Ignored someone in need");
      setState(STATES.S2B);
    }
    if (this.back?.hovering) setState(STATES.TITLE);
  },
};
```

---

scenes/scene2.js

```
const Scene2A = {
  draw() {
    drawHeader("SCENE 2A: THE CALL", "Help costs time. Time changes
consequences.");
    drawStatBar();

    const panel = { x: 36, y: 130, w: 828, h: 310 };
    drawPanel(panel.x, panel.y, panel.w, panel.h);

    const lines = [
      "You make the call. The stranger exhales like they were holding their
breath for hours.",
      "The guard notices. “Curfew. Nobody enters.”",
      "",
      "The stranger offers: “Tell him you’re with me. Say it’s an
emergency.”",
      "You can lie… or you can take the refusal and find another way."
    ];
    drawBodyText(lines, panel.x + 24, panel.y + 22, panel.w - 48);

    this.b1 = drawButton("Lie to the guard (push entry)", width / 2, 338,
520, 46, false);
    this.b2 = drawButton("Tell the truth (accept refusal)", width / 2, 398,
520, 46, false);
  },

  mousePressed() {
    if (this.b1?.hovering) {
      if (!game.flags.liedToGuard) {
        game.flags.liedToGuard = true;
        addKarma(-2, "Lied to the guard");
```

```
      }
      setState(STATES.S3A);
    }
    if (this.b2?.hovering) {
      addKarma(+1, "Accepted refusal without manipulation");
      setState(STATES.S3B);
    }
  },
};

const Scene2B = {
  draw() {
    drawHeader("SCENE 2B: THE GUARD", "Authority is a wall. Walls have weak
points.");
    drawStatBar();

    const panel = { x: 36, y: 130, w: 828, h: 310 };
    drawPanel(panel.x, panel.y, panel.w, panel.h);

    const lines = [
      "The guard finally speaks: “Curfew. Go home.”",
      "He lowers his voice: “Unless you make it worth my time.”",
      "",
      "A bribe opens the gate. Refusing means finding another entrance."
    ];
    drawBodyText(lines, panel.x + 24, panel.y + 22, panel.w - 48);

    this.b1 = drawButton("Pay the bribe", width / 2, 338, 520, 46, false);
    this.b2 = drawButton("Refuse and look for a side door", width / 2, 398,
520, 46, false);
  },

  mousePressed() {
    if (this.b1?.hovering) {
      if (!game.flags.tookBribe) {
        game.flags.tookBribe = true;
        addKarma(-2, "Bribed the guard");
      }
      setState(STATES.S3A);
    }
    if (this.b2?.hovering) {
      addKarma(+1, "Refused corruption");
      setState(STATES.S3B);
    }
  },
};
```

---

scenes/scene3.js

```
const Scene3A = {
  draw() {
    drawHeader("SCENE 3A: INSIDE", "Fast access, messy cost.");
    drawStatBar();
```

```
    const panel = { x: 36, y: 130, w: 828, h: 310 };
    drawPanel(panel.x, panel.y, panel.w, panel.h);

    const lines = [
      "You’re inside. The lights flicker like they’re deciding whether to
stay on.",
      "On the stairs, someone trips and drops a wallet. They don’t notice.",
      "",
      "You can return it… or keep it. Nobody saw."
    ];
    drawBodyText(lines, panel.x + 24, panel.y + 22, panel.w - 48);

    this.b1 = drawButton("Return the wallet", width / 2, 350, 520, 46,
false);
    this.b2 = drawButton("Keep it", width / 2, 410, 520, 46, false);
  },

  mousePressed() {
    if (this.b1?.hovering) {
      addKarma(+2, "Returned wallet");
      game.flags.savedFriend = true; // treat as a “good” flag for endings
      setState(STATES.END);
    }
    if (this.b2?.hovering) {
      addKarma(-3, "Kept wallet");
      setState(STATES.END);
    }
  },
};

const Scene3B = {
  draw() {
    drawHeader("SCENE 3B: SIDE DOOR", "Slower route. Cleaner hands.");
    drawStatBar();

    const panel = { x: 36, y: 130, w: 828, h: 310 };
    drawPanel(panel.x, panel.y, panel.w, panel.h);

    const lines = [
      "You find a maintenance door. It’s locked with a keypad.",
      "A tired worker exits and pauses: “You lost?”",
      "",
      "You can ask for help honestly, or sneak in behind them."
    ];
    drawBodyText(lines, panel.x + 24, panel.y + 22, panel.w - 48);

    this.b1 = drawButton("Ask honestly for help", width / 2, 350, 520, 46,
false);
    this.b2 = drawButton("Sneak in behind them", width / 2, 410, 520, 46,
false);
  },

  mousePressed() {
    if (this.b1?.hovering) {
      addKarma(+2, "Asked honestly");
      game.flags.savedFriend = true;
      setState(STATES.END);
```

```
    }
    if (this.b2?.hovering) {
      addKarma(-2, "Snuck in");
      setState(STATES.END);
    }
  },
};
```

---

scenes/endings.js

```
const EndingsScene = {
  draw() {
    drawHeader("ENDING", "Unlocked by karma + key choices.");
    drawStatBar();

    const panel = { x: 36, y: 130, w: 828, h: 310 };
    drawPanel(panel.x, panel.y, panel.w, panel.h);

    const ending = computeEnding();

    const lines = [
      ending.title,
      "",
      ending.text,
      "",
      `Key facts: helpedStranger=${game.flags.helpedStranger} |
liedToGuard=${game.flags.liedToGuard} | bribe=${game.flags.tookBribe} |
goodAct=${game.flags.savedFriend}`,
    ];
    drawBodyText(lines, panel.x + 24, panel.y + 22, panel.w - 48);

    this.b1 = drawButton("PLAY AGAIN", width / 2, 460, 240, 44, true);
    this.b2 = drawButton("TITLE", 820, 46, 120, 34, false);
  },

  mousePressed() {
    if (this.b1?.hovering) {
      resetGame();
      setState(STATES.S1);
    }
    if (this.b2?.hovering) {
      resetGame();
    }
  },
};

function computeEnding() {
  // Endings are intentionally gated by both stat and specific “ethical”
flags.
  // This makes the stat meaningful while still tying outcomes to decisions.
  const k = game.karma;

  // Best ending: high karma, and at least one explicit prosocial act.
  if (k >= 4 && game.flags.savedFriend) {
```

```
    return {
      id: "BEST",
      title: "THE GREEN LIGHT",
      text:
        "The station opens like it was waiting for you to choose decency. " +
        "A worker waves you through. Later, a message arrives: the person you
helped made it home. " +
        "You leave with nothing stolen, nothing owed, and no part of you
missing."
    };
  }

  // “Good” ending: positive karma without major integrity violations.
  if (k >= 1 && !game.flags.tookBribe && !game.flags.liedToGuard) {
    return {
      id: "GOOD",
      title: "THE SIDE ENTRANCE",
      text:
        "You get in the hard way. Slower, quieter, cleaner. " +
        "No shortcuts, no leverage. The night stays cold, but your decisions
don’t. " +
        "You reach the platform with your name intact."
    };
  }

  // “Neutral” ending: mixed karma and mixed behavior.
  if (k > -2 && k < 4) {
    return {
      id: "NEUTRAL",
      title: "THE FLICKER",
      text:
        "You make it through, but the story feels unfinished. " +
        "Small kindnesses and small compromises cancel each other out. " +
        "You arrive, but you can’t tell if you earned it or just survived
it."
    };
  }

  // Bad ending: low karma or explicit corruption/manipulation.
  if (k <= -2 || game.flags.tookBribe || game.flags.liedToGuard) {
    return {
      id: "BAD",
      title: "THE LOCKDOWN",
      text:
        "The gate opens, but it isn’t freedom—it’s a receipt. " +
        "Your shortcuts follow you like footsteps in wet cement. " +
        "By the time you reach the platform, you’re not late anymore. You’re
compromised."
    };
  }

  // Fallback
  return {
    id: "FALLBACK",
    title: "THE PLATFORM",
    text:
      "You arrive. The night continues. The choices remain yours next time."
```

```
  };
}
```

Ria

Give me a description of this game

Chat

The Silent Station is a short interactive narrative game built around choice, consequence, and ethical tension. The player navigates a closed transit station after curfew, making decisions that branch the story and incrementally alter a tracked karma stat. Each interaction—helping, lying, refusing, or exploiting—affects both immediate outcomes and the final ending. Endings are unlocked through a combination of accumulated karma and specific moral actions, emphasizing that how progress is achieved matters as much as reaching it.

Ria

The game doesn't work past the first screen

Chat

Likely cause: after you click START, the code switches game.state to SCENE_1, then the next draw() tries to run Scene1.draw(). If scenes/scene1.js failed to load (wrong path, wrong filename, wrong folder case), Scene1 is undefined, p5 throws an error, and the sketch stops updating. It looks like "stuck on the first screen."

Fix checklist (do these exactly)

1. **Open DevTools console**

- Chrome: View → Developer → JavaScript Console
- You will see an error like: ReferenceError: Scene1 is not defined or a 404 network error.

2. **Verify filenames and case match exactly**

   Your HTML expects:

- scenes/scene1.js
- scenes/scene2.js
- scenes/scene3.js
- scenes/endings.js

If your folder is Scenes/ (capital S) or file is Scene1.js or scene-1.js, it will break.

3. **Verify the script paths in index.html**

   These lines must match your actual structure:

```
<script defer src="scenes/scene1.js"></script>
<script defer src="scenes/scene2.js"></script>
<script defer src="scenes/scene3.js"></script>
<script defer src="scenes/endings.js"></script>
```

4. **Run from a local server (not by double-clicking HTML)**

   Use one of these:

- VS Code Live Server
- python3 -m http.server in the project folder

Add a hard failsafe so it never "silently freezes"

Replace currentScene() in stateManager.js with this:

```
function currentScene() {
  const missing = (name) => ({
    draw() {
      background(20);
      fill(255);
      textSize(18);
      textAlign(LEFT, TOP);
      text(`Missing scene: ${name}\nCheck file path + filename + folder
case.`, 24, 24);
      textSize(12);
      text(`Current state: ${game.state}`, 24, 80);
    },
```

```
  });

  switch (game.state) {
    case STATES.TITLE: return typeof TitleScene !== "undefined" ? TitleScene
: missing("TitleScene");
    case STATES.S1:    return typeof Scene1 !== "undefined" ? Scene1 :
missing("Scene1 (scenes/scene1.js)");
    case STATES.S2A:   return typeof Scene2A !== "undefined" ? Scene2A :
missing("Scene2A (scenes/scene2.js)");
    case STATES.S2B:   return typeof Scene2B !== "undefined" ? Scene2B :
missing("Scene2B (scenes/scene2.js)");
    case STATES.S3A:   return typeof Scene3A !== "undefined" ? Scene3A :
missing("Scene3A (scenes/scene3.js)");
    case STATES.S3B:   return typeof Scene3B !== "undefined" ? Scene3B :
missing("Scene3B (scenes/scene3.js)");
    case STATES.END:   return typeof EndingsScene !== "undefined" ?
EndingsScene : missing("EndingsScene (scenes/endings.js)");
    default:           return typeof TitleScene !== "undefined" ? TitleScene
: missing("TitleScene");
  }
}
```

This will display an on-canvas error telling you exactly which scene file didn't load.

Most common single fix: rename your folder/files to exactly scenes/scene1.js etc (lowercase) and run using Live Server.

Ria

After the first screen I press play and then the screen after that any option I press nothing follows

Chat

Root cause: your global p5 input handler is being overridden by another file. In p5 "global mode," only the last mousePressed() definition loaded wins. If any other file defines mousePressed() (common in other assignments), it replaces the one in main.js, so your scene buttons stop advancing.

Fix

1. Delete or rename every other mousePressed() in the project (search the whole folder for function mousePressed).
2. Keep exactly one global input handler in main.js.

Use this main.js input block (robust: handles click + touch, and doesn't rely on button .hovering state):

```
function mousePressed() {
  sceneMousePressed();
  return false;
}

function mouseClicked() {
  // some browsers fire clicked more reliably than pressed
  sceneMousePressed();
  return false;
}

function touchStarted() {
  sceneMousePressed();
  return false;
}
```

Also make button clicks reliable even if hovering state isn't updated

Replace every scene mousePressed() logic that checks this.b1?.hovering with direct hit-testing:

Example (do this pattern everywhere):

```
mousePressed() {
  if (this.b1 && isHover(this.b1)) {
    // action 1
    setState(STATES.S2A);
  }
  if (this.b2 && isHover(this.b2)) {
    // action 2
    setState(STATES.S2B);
  }
}
```

Reason: your current code checks this.b1.hovering, which is computed during draw(). If draw() stops (error) or the object isn't refreshed, clicks do nothing. isHover(this.b1) recomputes instantly at click time.

Hard fail indicator (so you never get a "silent" break)

Add this at the top of sceneMousePressed() in stateManager.js:

```
function sceneMousePressed() {
  console.log("CLICK state =", game.state);
  const s = currentScene();
  if (s && typeof s.mousePressed === "function") s.mousePressed();
}
```

If the console prints clicks but state never changes, you still have hover-check code somewhere. If it prints nothing, mousePressed is still being overridden by another file.

Ria

Still not working

Chat

Your global mousePressed() is still not firing, or it fires but your scene handler never receives it. Remove reliance on global p5 callbacks entirely and bind clicks directly to the canvas element. This bypasses every override problem.

Do this exactly

1) Delete these functions everywhere in the project

Remove all occurrences of:

- function mousePressed()
- function mouseClicked()
- function touchStarted()

Leave scene mousePressed() methods inside scene objects; those are fine.

2) Replace

main.js

with this

```
let cnv;

function setup() {
  cnv = createCanvas(900, 520);
  textFont("system-ui");
  noSmooth();

  // Bind input directly to the canvas element (cannot be overridden by other
files)
  cnv.mousePressed(() => sceneMousePressed());
  cnv.touchStarted(() => sceneMousePressed());
}

function draw() {
  background(11, 13, 18);

  // subtle grid
  push();
  stroke(255, 255, 255, 10);
  for (let x = 0; x < width; x += 24) line(x, 0, x, height);
  for (let y = 0; y < height; y += 24) line(0, y, width, y);
  pop();

  sceneDraw();
}
```

3) Make clicks independent of “hovering” state

In every scene file, change click checks from:

```
if (this.b1?.hovering) { ... }
```

to:

```
if (this.b1 && isHover(this.b1)) { ... }
```

Example patch (apply this pattern everywhere):

```
mousePressed() {
  if (this.b1 && isHover(this.b1)) {
    setState(STATES.S2A);
  }
  if (this.b2 && isHover(this.b2)) {
    setState(STATES.S2B);
  }
```

```
}
```

4) Add a forced on-screen debug readout (no console needed)

Add this at the end of sceneDraw() in stateManager.js:

```
function sceneDraw() {
  currentScene().draw();

  // Debug overlay
  push();
  noStroke();
  fill(0, 0, 0, 140);
  rect(10, height - 40, 360, 30, 8);
  fill(255);
  textSize(12);
  textAlign(LEFT, CENTER);
  text(`state=${game.state}  karma=${game.karma}
mouse=(${floor(mouseX)},${floor(mouseY)})`, 18, height - 25);
  pop();
}
```

If this still “does nothing,” the only remaining failure mode is that your scene files aren’t loading (404 / wrong folder case). The debug overlay will still display, and the state will stay stuck on SCENE_1 when you click.

Ria

Restart from stratch and make a whole new game

Chat

File tree

```
new-game/
  index.html
  style.css
  main.js
  engine.js
  ui.js
  story.js
```

---

index.html

```
<!doctype html>
```

```
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width,initial-scale=1" />
    <title>Decision Tree Story</title>
    <link rel="stylesheet" href="style.css" />
    <script
src="https://cdn.jsdelivr.net/npm/p5@1.9.0/lib/p5.min.js"></script>

    <script defer src="story.js"></script>
    <script defer src="ui.js"></script>
    <script defer src="engine.js"></script>
    <script defer src="main.js"></script>
  </head>
  <body>
    <main></main>
  </body>
</html>
```

---

style.css

```
html, body {
  margin: 0;
  padding: 0;
  background: #0b0d12;
  color: #e9eefc;
  font-family: system-ui, -apple-system, Segoe UI, Roboto, Arial, sans-serif;
}

canvas {
  display: block;
  margin: 0 auto;
}
```

---

story.js

```
// Stat: TRUST (player earns/loses it)
// Endings unlock based on trust threshold + key flags.

const STORY = {
  startNode: "TITLE",

  nodes: {
    TITLE: {
      title: "THE LOST USB",
      body: [
        "You find a USB on campus.",
        "A label reads: “CO-OP APPLICATIONS — FINAL.””,
        "Trust is tracked. Choices branch. Endings unlock by trust."
      ],
      choices: [
        { label: "Start", next: "FOUND" },
      ],
    },
```

```
    FOUND: {
      title: "FOUND",
      body: [
        "It’s late. The lab is quiet.",
        "You can turn it in now, or open it first.",
        "What do you do?"
      ],
      choices: [
        { label: "Turn it into the front desk", next: "DESK", trust: +2,
flag: "returnedEarly" },
        { label: "Plug it in and check what’s inside", next: "OPEN", trust: -
2, flag: "snooped" },
      ],
    },

    DESK: {
      title: "DESK",
      body: [
        "The desk staff logs it, but asks:",
        "“Any idea whose it is?”",
        "You can help identify the owner, or leave it anonymous."
      ],
      choices: [
        { label: "Stay and help identify owner", next: "OWNER", trust: +1,
flag: "helpedDesk" },
        { label: "Leave it and go", next: "NIGHT", trust: 0 },
      ],
    },

    OPEN: {
      title: "OPEN",
      body: [
        "You open a folder: resumes, cover letters, transcripts.",
        "There’s also a doc called: “Interview Qs — internal notes”.",
        "You can stop now, or keep digging."
      ],
      choices: [
        { label: "Stop. Close everything and turn it in", next: "DESK",
trust: +1, flag: "stoppedSnooping" },
        { label: "Copy files to your laptop", next: "COPY", trust: -3, flag:
"copied" },
      ],
    },

    COPY: {
      title: "COPY",
      body: [
        "You copied the files. Nobody saw.",
        "Your friend texts: “Need help polishing my application tonight.”",
        "Do you use what you took?"
      ],
      choices: [
        { label: "Delete the copy. Never use it", next: "DESK", trust: +2,
flag: "deletedCopy" },
        { label: "Use it as “inspiration”", next: "USE", trust: -2, flag:
"usedIt" },
```

```
      ],
    },

    USE: {
      title: "USE",
      body: [
        "You reuse phrasing. It reads cleaner.",
        "A week later, you recognize the owner in class.",
        "They mention their USB went missing. They look exhausted."
      ],
      choices: [
        { label: "Confess privately", next: "CONFESS", trust: +1, flag:
"confessed" },
        { label: "Say nothing", next: "SILENT", trust: -2, flag: "hidIt" },
      ],
    },

    OWNER: {
      title: "THE OWNER",
      body: [
        "You spot the owner’s name on a file header without opening it.",
        "It’s someone in your program.",
        "You can message them, or let the desk handle it."
      ],
      choices: [
        { label: "Message them: “I turned in your USB.”", next: "MEET",
trust: +2, flag: "messagedOwner" },
        { label: "Let desk handle it", next: "NIGHT", trust: 0 },
      ],
    },

    MEET: {
      title: "MEETUP",
      body: [
        "They meet you to pick it up.",
        "They thank you and offer you coffee as a thank-you.",
        "Do you accept?"
      ],
      choices: [
        { label: "Accept coffee (friendly)", next: "END_GOOD", trust: +1,
flag: "acceptedCoffee" },
        { label: "Decline politely and leave", next: "END_GOOD", trust: 0 },
      ],
    },

    NIGHT: {
      title: "NIGHT WALK",
      body: [
        "You leave the building.",
        "You replay the moment in your head: you could’ve done more, or
less.",
        "Your next choice decides what kind of ending you get."
      ],
      choices: [
        { label: "Check the desk later to confirm it was returned", next:
"CHECKBACK", trust: +1, flag: "checkedBack" },
        { label: "Forget it and move on", next: "END_NEUTRAL", trust: 0 },
```

```
      ],
    },

    CHECKBACK: {
      title: "CHECKBACK",
      body: [
        "The desk says: “Owner picked it up.”",
        "You feel lighter.",
        "This ends well if your trust is high enough."
      ],
      choices: [
        { label: "Finish", next: "END_GOOD", trust: 0 },
      ],
    },

    CONFESS: {
      title: "CONFESSION",
      body: [
        "You admit you opened it and copied files.",
        "They go quiet, then ask: “Did you use anything?”",
        "Your answer locks the ending."
      ],
      choices: [
        { label: "Tell the full truth", next: "END_REPAIR", trust: +2, flag:
"fullTruth" },
        { label: "Lie: “No.”", next: "END_BAD", trust: -3, flag: "lied" },
      ],
    },

    SILENT: {
      title: "SILENCE",
      body: [
        "You keep it to yourself.",
        "Later, your application gets flagged for “similarity.”",
        "You’re asked to explain."
      ],
      choices: [
        { label: "Own it", next: "END_BAD", trust: -1 },
        { label: "Deny it", next: "END_WORST", trust: -2 },
      ],
    },

    // End nodes
    END_GOOD: {
      title: "ENDING: CLEAN HANDS",
      body: [
        "You chose restraint when you could’ve taken advantage.",
        "The story ends with your reputation intact.",
        "Trust tends to compound."
      ],
      ending: "GOOD",
      choices: [{ label: "Play again", next: "TITLE", reset: true }],
    },

    END_NEUTRAL: {
      title: "ENDING: NOTHING HAPPENS",
      body: [
```

```
        "You didn’t harm anyone directly, but you also didn’t follow
through.",
        "Life moves on.",
        "So does the pattern."
      ],
      ending: "NEUTRAL",
      choices: [{ label: "Play again", next: "TITLE", reset: true }],
    },

    END_REPAIR: {
      title: "ENDING: REPAIR",
      body: [
        "You told the truth even when it cost you.",
        "They don’t forgive you instantly, but they believe you’re not
pretending.",
        "Trust can be rebuilt, slowly, with action."
      ],
      ending: "REPAIR",
      choices: [{ label: "Play again", next: "TITLE", reset: true }],
    },

    END_BAD: {
      title: "ENDING: CONSEQUENCES",
      body: [
        "Shortcuts became evidence.",
        "You lose opportunities you didn’t even know you were being
considered for.",
        "Trust collapses faster than it forms."
      ],
      ending: "BAD",
      choices: [{ label: "Play again", next: "TITLE", reset: true }],
    },

    END_WORST: {
      title: "ENDING: BURNED",
      body: [
        "You doubled down on denial.",
        "Now it’s not just what you did — it’s who you decided to be about
it.",
        "Doors close quietly and stay closed."
      ],
      ending: "WORST",
      choices: [{ label: "Play again", next: "TITLE", reset: true }],
    },
  },
};
```

---

ui.js

```
function panel(x, y, w, h) {
  push();
  noStroke();
  fill(18, 22, 33, 235);
  rect(x, y, w, h, 18);
  pop();
```

```
}

function header(title, subtitle) {
  push();
  fill(233, 238, 252);
  textAlign(LEFT, TOP);
  textStyle(BOLD);
  textSize(28);
  text(title, 34, 26);

  textStyle(NORMAL);
  textSize(14);
  fill(185, 196, 220);
  text(subtitle, 34, 60);
  pop();
}

function statBar(trust) {
  const x = 34, y = 92, w = 320, h = 12;
  const t = constrain(trust, -10, 10);

  push();
  noStroke();
  fill(255, 255, 255, 18);
  rect(x, y, w, h, 8);

  const mid = x + w / 2;
  const px = map(t, -10, 10, x, x + w);

  fill(255, 120, 120, 140);
  rect(min(px, mid), y, abs(px - mid), h, 8);

  fill(120, 255, 190, 140);
  rect(min(px, mid), y, abs(px - mid), h, 8);

  stroke(255, 255, 255, 40);
  line(mid, y - 2, mid, y + h + 2);

  noStroke();
  fill(233, 238, 252);
  textSize(12);
  textAlign(LEFT, BOTTOM);
  text(`TRUST: ${trust}`, x, y - 6);
  pop();
}

function wrapTextLines(lines, x, y, w) {
  push();
  fill(233, 238, 252);
  textAlign(LEFT, TOP);
  textSize(16);

  let yy = y;
  for (const line of lines) {
    text(line, x, yy, w);
    yy += 24;
  }
```

```
  pop();
}

function makeButtonRect(cx, cy, w, h) {
  return { x: cx - w / 2, y: cy - h / 2, w, h };
}

function pointInRect(px, py, r) {
  return px >= r.x && px <= r.x + r.w && py >= r.y && py <= r.y + r.h;
}

function drawButton(r, label) {
  const hover = pointInRect(mouseX, mouseY, r);
  push();
  noStroke();
  fill(hover ? color(90, 120, 255, 210) : color(255, 255, 255, 18));
  rect(r.x, r.y, r.w, r.h, 14);

  fill(233, 238, 252);
  textAlign(CENTER, CENTER);
  textSize(14);
  text(label, r.x + r.w / 2, r.y + r.h / 2);
  pop();
  return hover;
}
```

---

engine.js

```
const model = {
  nodeId: STORY.startNode,
  trust: 0,
  flags: {},
};

function resetModel() {
  model.nodeId = STORY.startNode;
  model.trust = 0;
  model.flags = {};
}

function getNode() {
  const n = STORY.nodes[model.nodeId];
  if (!n) {
    // Hard fail-safe node
    return {
      title: "ERROR",
      body: [`Missing node: ${model.nodeId}`],
      choices: [{ label: "Reset", next: STORY.startNode, reset: true }],
    };
  }
  return n;
}

function applyChoice(choice) {
  if (choice.reset) resetModel();
```

```
  if (typeof choice.trust === "number") model.trust += choice.trust;

  if (choice.flag) model.flags[choice.flag] = true;

  if (choice.next) model.nodeId = choice.next;

  // Optional gating: if you want trust-based locks, do it here.
}

function drawNode() {
  const node = getNode();

  header(node.title, "Branching decision-tree story. Stat tracked across
scenes.");
  statBar(model.trust);

  const p = { x: 34, y: 126, w: 832, h: 320 };
  panel(p.x, p.y, p.w, p.h);

  wrapTextLines(node.body, p.x + 22, p.y + 20, p.w - 44);

  // Buttons
  const btns = [];
  const startY = 360;
  const gap = 56;
  const bw = 620;
  const bh = 44;

  for (let i = 0; i < node.choices.length; i++) {
    const cy = startY + i * gap;
    const r = makeButtonRect(width / 2, cy, bw, bh);
    btns.push({ r, choice: node.choices[i] });
  }

  // Draw and store for click handling
  engineRuntime.buttons = btns;
  for (const b of btns) drawButton(b.r, b.choice.label);

  // Debug overlay (always visible)
  push();
  noStroke();
  fill(0, 0, 0, 140);
  rect(12, height - 36, 520, 26, 8);
  fill(255);
  textSize(12);
  textAlign(LEFT, CENTER);
  text(`node=${model.nodeId}  trust=${model.trust}
flags=${Object.keys(model.flags).length}`, 20, height - 23);
  pop();
}

const engineRuntime = {
  buttons: [],
};

function handleClick(mx, my) {
```

```
  for (const b of engineRuntime.buttons) {
    if (pointInRect(mx, my, b.r)) {
      applyChoice(b.choice);
      return;
    }
  }
}
```

---

main.js

```
let cnv;

function setup() {
  cnv = createCanvas(900, 520);
  textFont("system-ui");

  // Bind input directly to canvas: cannot be overridden by other files
  cnv.mousePressed(() => handleClick(mouseX, mouseY));
  cnv.touchStarted(() => {
    handleClick(mouseX, mouseY);
    return false;
  });
}

function draw() {
  background(11, 13, 18);

  // subtle grid
  push();
  stroke(255, 255, 255, 10);
  for (let x = 0; x < width; x += 24) line(x, 0, x, height);
  for (let y = 0; y < height; y += 24) line(0, y, width, y);
  pop();

  drawNode();
}
```

Ria

Make description of game

Chat

The Lost USB is a branching interactive story about ethics, restraint, and trust in an academic setting. The player finds a misplaced USB containing sensitive co-op application materials and must decide how to handle it. Each choice branches the narrative and alters a tracked **trust** stat, shaping both immediate consequences and long-term outcomes. Endings are unlocked based on accumulated trust and key moral decisions, emphasizing that integrity is defined not just by actions, but by how temptation is handled when no one is watching.